

IdeaBee

AI-Powered Startup Builder

5-Agent System — Full Project Blueprint

5

AI Agents

RAG

Powered Research

100%

Free APIs

1. Project Overview

What is IdeaBee?

IdeaBee is a 5-agent AI system that turns a raw startup idea into a complete package: a live deployed website, an investor-ready pitch deck, and a professional pitch speech — all generated automatically using free LLMs, RAG, and open-source tools.

Core Value Proposition

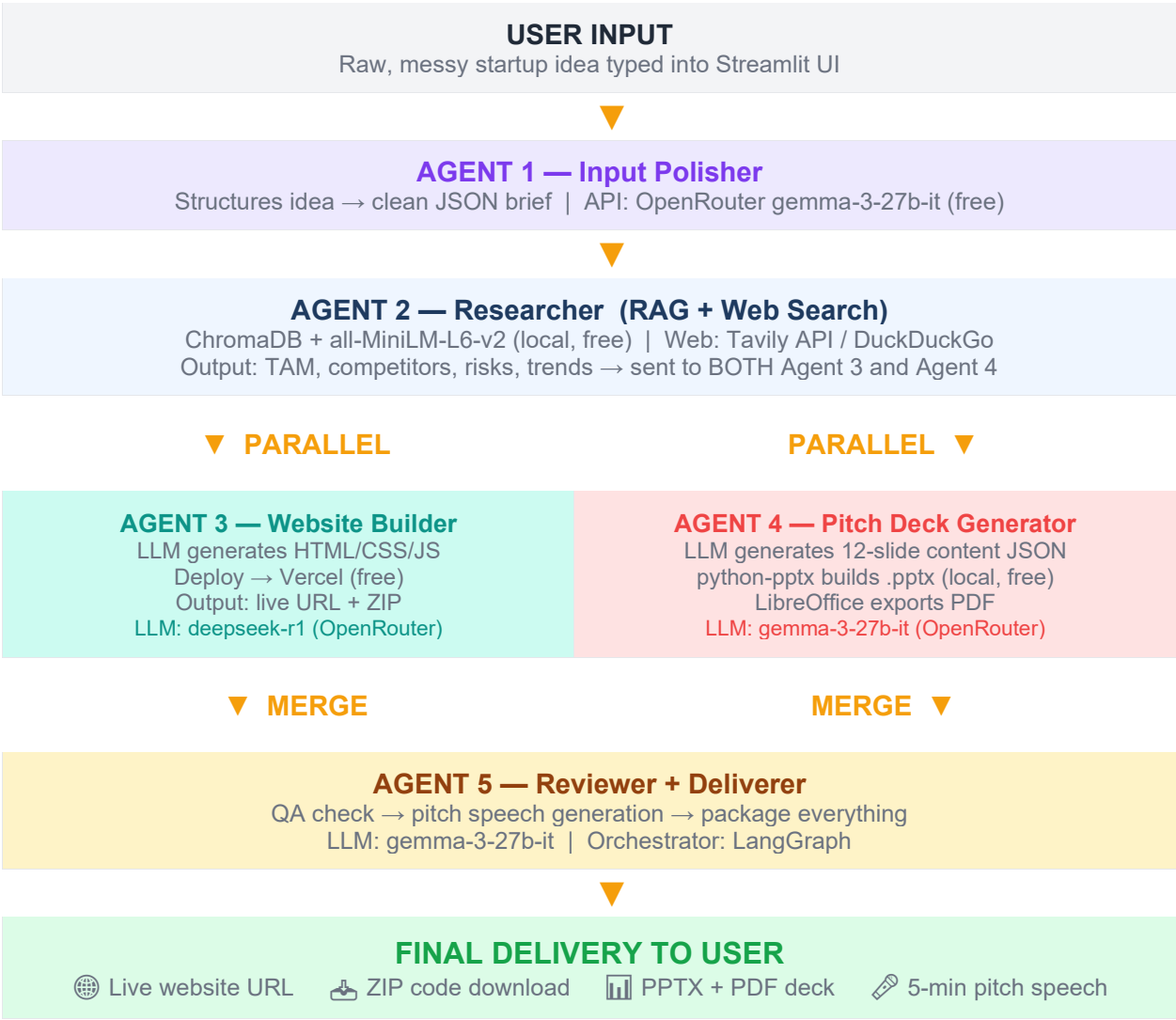
A student or early-stage founder types their messy idea — IdeaBee's agents collaborate to research it, build a website, create a pitch deck, and deliver everything in one package.

Key Outputs

- Live deployed website URL (via Vercel)
- Downloadable website source code as ZIP
- 12-slide investor-ready pitch deck (.pptx + .pdf)
- 5-minute professional pitch speech script
- Market research report with TAM, competitors, risks

2. System Architecture & Flowchart

The full IdeaBee pipeline runs sequentially with a parallel branch after Agent 2. Agent 3 and Agent 4 run simultaneously, then merge at Agent 5 for final delivery.



3. Agent-by-Agent Breakdown

Agent 1 — Input Polisher

Purpose

Takes any raw, messy user idea and converts it into a clean, structured JSON brief that all downstream agents can use.

Flowchart

Raw user text → LLM system prompt (structure this into JSON) → Parse JSON response → Valid? → Yes: pass to Agent 2 / No: wrap as {raw_summary: ...} → Agent 2

Free API

Item	Value	Note
LLM	google/gemma-3-27b-it	Free tier on OpenRouter
Platform	openrouter.ai	No credit card needed
Alt LLM	mistralai/mistral-7b-instruct	HuggingFace Inference API

Output JSON structure

- startup_name, elevator_pitch, problem, solution
- target_audience, key_features[], uvp
- suggested_tech_stack

Agent 2 — Researcher (RAG + Web Search)

Purpose

Deep-dives into the polished brief using RAG retrieval from a local knowledge base and live web search. Produces a comprehensive market research report.

Flowchart

Polished brief → SPLIT:

- RAG path: ChromaDB vector search → top-k chunks from knowledge base
- Web path: Tavily API / DuckDuckGo → live 2026 data

Both paths MERGE → LLM synthesis (OpenRouter) → Research JSON → sent to Agent 3 AND Agent 4

Free APIs & Tools

Task	Tool / API	Cost
Embedding model	all-MiniLM-L6-v2 (HuggingFace)	Free, runs locally
Vector store	ChromaDB (local)	Free, saves to disk
Chunking	RecursiveCharacterTextSplitter	LangChain, free
Web search	Tavily API	1000 free calls/month
Alt search	DuckDuckGo Search	No API key needed
LLM synthesis	google/gemma-3-27b-it	OpenRouter free tier

Research output includes

- Market size (TAM/SAM/SOM) with CAGR
- Top 3-5 competitors with strengths and weaknesses
- Technical feasibility and development timeline
- Revenue model suggestions
- Top 3 risks with mitigation strategies
- Recent 2026 trends

Agent 3 — Website Builder + Deployment

Purpose
Uses the brief and research to generate a complete, professional website (HTML/CSS/JS), saves files to disk, creates a ZIP, and auto-deploys to Vercel.

Flowchart

Brief + Research → LLM generates website code (deepseek-r1) → Parse response → Write files to disk (index.html, styles.css, script.js, README.md) → Create ZIP archive (Python zipfile) → subprocess('vercel --prod') → Returns live URL → Output to Agent 5

Free APIs & Tools

Task	Tool / API	Cost
Code LLM	deepseek/deepseek-r1	OpenRouter free tier
Alt code LLM	qwen/qwen-2.5-coder-32b-instruct	OpenRouter free tier
Hosting	Vercel CLI	Free hobby tier
Alt hosting	Netlify CLI	Free tier
ZIP creation	Python zipfile (built-in)	No install needed

Website pages generated

- Hero section (startup name, tagline, CTA button)
- Features section (from key_features in brief)
- About / Problem section
- Pricing section
- Contact form
- SEO meta tags + responsive design

Agent 4 — Pitch Deck Generator

Purpose
Generates a 12-slide investor-ready pitch deck. LLM produces slide content as JSON; python-pptx (or a professional template) renders the actual .pptx file locally — no API cost for file creation.

Flowchart

Brief + Research → LLM generates 12-slide JSON (title, bullets, notes per slide) → Parse JSON → python-pptx renders .pptx using pre-designed template → LibreOffice CLI exports PDF → Output: startup_pitch.pptx + .pdf → Agent 5

Free Tools — Why python-pptx needs a template

python-pptx alone places text but has no design intelligence. The best approach: download a professional .pptx template free from Slidesgo or SlidesCarnival, load it as the base, and only replace text in placeholders. The design (fonts, colors, gradients) stays intact.

Task	Tool	Cost
Slide content	gemma-3-27b-it (OpenRouter)	Free
PPTX builder	python-pptx + free template	100% free, local
Templates	Slidesgo / SlidesCarnival	Free download (.pptx)
PDF export	LibreOffice CLI headless	Free, open-source
Alt PDF	reportlab (pip install)	Free
Canva API option	Canva Connect API	Free tier available

12 slides generated

- Slide 1: Title (name, tagline, contact)
- Slide 2: Problem (pain points, current failures)
- Slide 3: Solution (product overview, benefits)
- Slide 4: Market Opportunity (TAM/SAM/SOM, Why now)
- Slide 5: Product Demo (features, user flow)
- Slide 6: Business Model (revenue streams, pricing)
- Slide 7: Traction / Roadmap (milestones, 12-month plan)
- Slide 8: Competition (landscape, unfair advantage)
- Slide 9: Go-to-Market Strategy
- Slide 10: Team (founders, advisors)
- Slide 11: Financial Projections (3-year forecast)
- Slide 12: The Ask (funding amount, use of funds)

Agent 5 — Reviewer + Deliverer

Purpose

Receives outputs from Agents 3 and 4, performs quality checks, generates a 5-minute pitch speech, and packages everything into a single final delivery for the user.

Flowchart

Website URL + ZIP (Agent 3) + PPTX + PDF (Agent 4) → QA check (LLM verifies 12 slides, valid HTML, consistency) → Generate 5-min pitch speech (750 words, timed sections, Q&A prep) → Package everything into ZIP → Final delivery message to user

QA checks performed

- Verifies pitch deck has all 12 slides
- Checks HTML for basic syntax errors
- Confirms consistency between brief, website, and deck
- Flags missing or weak content
- Generates quality score (e.g. 9/10 — production ready)

Speech script structure

- [0:00-0:30] Opening — elevator pitch
- [0:30-1:30] Problem — pain points
- [1:30-2:30] Solution — product overview
- [2:30-3:30] Market + Traction
- [3:30-4:30] Business model + Competition
- [4:30-5:00] The Ask — funding round
- + 3 likely investor Q&A with prepared answers

4. RAG Pipeline — Full Build Guide

RAG (Retrieval-Augmented Generation) is what makes IdeaBee's research grounded in real data rather than LLM hallucinations. It is used in Agent 2 and optionally in Agent 1. The pipeline has two phases: build once (offline) and query every time Agent 2 runs.

Phase A — Build the Knowledge Base (Run Once)

Step	Action	Free Tool
1	Collect documents — PDFs, TXTs, web pages, markdown files	Manual download
2	Load documents — PyPDFLoader, WebBaseLoader (LangChain)	LangChain (free)
3	Chunk — RecursiveCharacterTextSplitter, size=1000, overlap=200	LangChain (free)
4	Embed — Convert chunks to vectors using embedding model	all-MiniLM-L6-v2 (HF, local)
5	Store — Save vectors + metadata to ChromaDB on disk	ChromaDB (free, local)

Phase B — Query at Runtime (Every Agent 2 Call)

Step	Action	Tool
6	Embed the query — same model as step 4	all-MiniLM-L6-v2
7	Similarity search — cosine distance, retrieve top-k=5 chunks	ChromaDB
8	Inject chunks into LLM system prompt as context	LangChain
9	LLM generates grounded, cited research output	OpenRouter (free)

Knowledge Base — What to Put In It

The quality of RAG output depends entirely on the quality of your knowledge base. All sources below are free to collect.

Category	Source	Why it helps
Market data	YC Library (ycombinator.com/library)	Proven frameworks, real examples

Startup failures	IndieHackers postmortems	Warns against common pitfalls
Pitch decks	Sequoia / a16z public guides	What investors expect
Market reports	Statista free previews, CB Insights	Real TAM numbers
Web best practices	Google Web Fundamentals, MDN Docs	Agent 3 code quality
Security	OWASP Top 10 (free PDF)	Agent 3 secure code

5. Complete Free API Stack

Every tool and API listed below has a free tier sufficient for a student project. No credit card required for any of the core tools.

Agent	Task	Tool / Model	Platform	Cost
Agent 1	LLM	google/gemma-3-27b-it	OpenRouter	Free
Agent 2	Embedding	all-MiniLM-L6-v2	HuggingFace local	Free
Agent 2	Vector DB	ChromaDB	Local	Free
Agent 2	Web search	Tavily API	tavily.com	1000/month
Agent 2	Alt search	DuckDuckGo	No key needed	Free
Agent 3	Code LLM	deepseek/deepseek-r1	OpenRouter	Free
Agent 3	Hosting	Vercel CLI	vercel.com	Free hobby
Agent 4	Content LLM	google/gemma-3-27b-it	OpenRouter	Free
Agent 4	PPTX builder	python-pptx + template	Local	Free
Agent 4	PDF export	LibreOffice headless	Local	Free
Agent 5	LLM	google/gemma-3-27b-it	OpenRouter	Free
All agents	Orchestration	LangGraph	pip install	Free
UI	Web interface	Streamlit	pip install	Free

OpenRouter free models (sign up at openrouter.ai — no payment needed)

google/gemma-3-27b-it • meta-llama/llama-3.3-70b-instruct • deepseek/deepseek-r1 • qwen/qwen-2.5-coder-32b-instruct • mistralai/mistral-7b-instruct

6. Full Tech Stack

Layer	Technology	Purpose
Orchestration	LangGraph	State management, parallel execution, agent graph
LLM (primary)	google/gemma-3-27b-it via OpenRouter	All agents — free tier
LLM (code)	deepseek/deepseek-r1 via OpenRouter	Agent 3 website generation
RAG framework	LangChain	Document loading, chunking, retrieval
Embeddings	all-MiniLM-L6-v2 (HuggingFace)	Runs locally, no API cost
Vector store	ChromaDB	Persists to disk, fully local
Web search	Tavily API / DuckDuckGo	Live 2026 market data
Website deploy	Vercel CLI	Free hobby hosting, auto-deploy via subprocess
PPTX builder	python-pptx + Slidesgo template	Fancy investor deck, no API cost
PDF export	LibreOffice CLI headless	Converts .pptx to .pdf locally
UI	Streamlit	Simple web interface for user input and results
Language	Python 3.11+	All agents and orchestration

7. Step-by-Step Build Order

Follow this exact sequence. Each step gives you something working before moving to the next.

#	Task	Milestone
1	Sign up at openrouter.ai, get free API key, test a simple LLM call in Python	LLM call works in terminal
2	Build Agent 1 alone — test with 5 different messy startup ideas	Clean JSON brief generated
3	Set up ChromaDB + download all-MiniLM-L6-v2 locally — embed 10 test docs	RAG retrieval working
4	Build Agent 2 with RAG + Tavily web search — test market research output	Research JSON generated
5	Build Agent 3 — generate HTML page, manually deploy to Vercel once	Live URL exists
6	Build Agent 4 — generate one PPTX with python-pptx using a template	.pptx file opens correctly
7	Build Agent 5 — QA check + speech generation + packaging	Full package assembled
8	Wire all 5 agents into LangGraph StateGraph — run end-to-end test	Full pipeline works
9	Build Streamlit UI — text area input, spinner, results display with download buttons	Web UI works
10	Deploy Streamlit app to Streamlit Cloud (free) — share the link	IdeaBee is live!

8. Folder Structure

Path	Purpose
ideabee/	Project root
main.py	LangGraph workflow entry point
app.py	Streamlit UI
agents/	One file per agent
agent1_polisher.py	Input polisher
agent2_researcher.py	RAG + web search researcher
agent3_website.py	Website generator + deployer
agent4_pitch.py	Pitch deck generator
agent5_deliverer.py	Reviewer + packager
rag/	RAG setup and query utilities
build_kb.py	Run once to build ChromaDB
retriever.py	Query interface for Agent 2
knowledge_base/	PDFs, TXTs, markdown docs for RAG
templates/	Pre-designed .pptx templates (from Slidesgo)
outputs/	Generated websites, decks, scripts
prompts/	System prompts for each agent
.env	API keys (OpenRouter, Tavily)
requirements.txt	All Python dependencies

requirements.txt

```
langgraph langchain langchain-openai langchain-community
chromadb sentence-transformers tavily-python
python-pptx reportlab pillow streamlit python-dotenv
beautifulsoup4 requests aiohttp groq
```

9. Why IdeaBee Stands Out

Skill demonstrated	How IdeaBee shows it
Multi-agent orchestration	LangGraph StateGraph with 5 nodes, parallel execution, shared state
RAG / Retrieval AI	ChromaDB + sentence-transformers + Tavily — grounded research output
Code generation	Agent 3 writes and deploys real websites end-to-end
Tool use	Web search, file I/O, subprocess Vercel CLI, ZIP creation
Real outputs	Live URL + downloadable deck + speech — not just text on screen
Business acumen	12-slide investor deck, market validation, TAM analysis
Full-stack thinking	Frontend (Streamlit), backend (Python agents), infra (Vercel deploy)

IdeaBee — Built by you, powered by free AI

Start with Agent 1. Test it. Then build the next one. Ship it.

May Help:-

COMPLETE 5-AGENT SYSTEM - FULL IMPLEMENTATION GUIDE

1. INSTALLATION & SETUP

bash

Create project folder

mkdir startup-builder-agent

cd startup-builder-agent

Create virtual environment

python -m venv venv

source venv/bin/activate *# On Windows: venv\Scripts\activate*

Install requirements

pip install langgraph langchain langchain-openai python-dotenv \

aiohttp requests beautifulsoup4 python-pptx reportlab pillow \

groq streamlit

2. AGENT PROMPTS (Copy-Paste Ready)

Agent 1: Input Polisher / Clarifier

python

POLISHER_PROMPT = """

You are an expert Startup Idea Clarifier. Your job is to transform messy, vague user ideas into clear, professional, actionable project briefs.

INPUT: Raw user idea (may be unstructured, ambiguous, or incomplete)

YOUR TASK:

1. Identify the core business/problem being solved
2. Clarify ambiguous terms and assumptions
3. Structure into this format:
 - Startup Name (suggest if not provided)
 - One-sentence elevator pitch
 - Problem statement (2-3 sentences)
 - Solution overview (2-3 sentences)
 - Target audience
 - Key features (bullet list, 3-5 items)
 - Unique value proposition
 - Suggested tech stack (if applicable)
4. Remove redundancies and improve flow
5. Keep it concise but comprehensive (max 400 words)

OUTPUT FORMAT (JSON):

```
{{  
  "startup_name": "...",  
  "elevator_pitch": "...",  
  "problem": "...",
```



```

"solution": "...",
"target_audience": "...",
"key_features": ["...", "...", "..."],
"uvp": "...",
"suggested_tech_stack": "..."
}}

```

TONE: Professional, clear, investor-ready
 ""

Agent 2: Researcher

python

RESEARCHER_PROMPT = ""

You are an expert Market Research Analyst. Your job is to gather comprehensive data about the startup idea to make it investor-ready.

INPUT: Polished startup brief from Agent 1

YOUR TASK:

1. Market Analysis:
 - Total Addressable Market (TAM) with numbers
 - Growth rate and trends
 - Key market drivers
2. Competitor Analysis:
 - Top 3-5 direct competitors
 - Their strengths and weaknesses
 - Market gaps your startup fills
3. Technical Feasibility:
 - Required technologies and their maturity
 - Estimated development timeline
 - Potential technical challenges
4. Business Model Suggestions:
 - Revenue model options (SaaS, subscription, freemium, etc.)
 - Pricing strategy suggestions
 - Customer acquisition cost estimates
5. Risk Assessment:
 - Top 3 risks and mitigation strategies
6. Recent News/Trends (last 6 months) related to this space

OUTPUT FORMAT (JSON):

```

{{
  "market_size": "{{tam}}, {{cagr}} growth",
  "competitors": [

```

```

    {{name: "...", strength: "...", weakness: "..."}},
    ...
  ],
  "technical_requirements": "...",
  "development_timeline": "...",
  "revenue_models": ["...", "..."],
  "risks": [
    {{risk: "...", mitigation: "..."}}
    ...
  ],
  "recent_trends": ["...", "..."]
}}

```

TOOLS: Use web_search to find current data, stats, and competitor info
 """

Agent 3: Website Builder

python

WEBSITE_BUILDER_PROMPT = """

You are an expert Full-Stack Developer. Your job is to create a complete, deployable website for the startup.

INPUT: Polished brief + Research data

YOUR TASK:

1. Generate complete website code:
 - HTML/CSS/JavaScript (or React/Next.js if complex)
 - Responsive design (mobile-first)
 - Modern, professional UI/UX
 - Include: Hero section, Features, About, Pricing, Contact
2. Create all necessary files:
 - index.html
 - styles.css
 - script.js
 - package.json (if using Node.js)
 - README.md with deployment instructions
3. Add placeholder content based on the startup brief
4. Include proper meta tags, SEO optimization
5. Make it production-ready (clean code, comments)
6. Generate deployment script for Vercel/Netlify

OUTPUT STRUCTURE:

```

{{
  "files": {{
    "index.html": "full code...",

```

```
"styles.css": "full code...",
"script.js": "full code...",
"package.json": "full code...",
"README.md": "deployment instructions..."
}},
"deployment_url_suggestion": "vercel deploy --prod",
"estimated_build_time": "15-20 minutes"
}}
```

TECH STACK: Use modern, simple stack (HTML/CSS/JS or Next.js)
"""

Agent 4: Pitch Deck Generator

python

PITCH_DECK_PROMPT = """

You are an expert Pitch Deck Consultant (ex-Y Combinator). Your job is to create an investor-ready 12-slide pitch deck.

INPUT: Polished brief + Research data

YOUR TASK:

Create slides with this exact structure:

Slide 1: Title Slide

- Startup name, tagline, contact info

Slide 2: Problem

- Clear problem statement
- Pain points with data
- Current solutions' failures

Slide 3: Solution

- Your product/service
- How it solves the problem
- Key benefits

Slide 4: Market Opportunity

- TAM/SAM/SOM numbers
- Market growth rate
- Why now?

Slide 5: Product Demo

- Screenshots/features showcase
- User flow
- Key differentiators

Slide 6: Business Model

- Revenue streams

- Pricing strategy
- Unit economics

Slide 7: Traction/Roadmap

- Current progress (if any)
- Milestones achieved
- 12-month roadmap

Slide 8: Competition

- Competitive landscape
- Your unfair advantage
- Comparison matrix

Slide 9: Go-to-Market Strategy

- Customer acquisition channels
- Marketing strategy
- Sales funnel

Slide 10: Team

- Founders & key roles
- Relevant experience
- Advisors (if any)

Slide 11: Financial Projections

- 3-year revenue forecast
- Key metrics
- Break-even analysis

Slide 12: The Ask

- Funding amount needed
- Use of funds breakdown
- What you're offering

OUTPUT: Generate python-pptx code to create actual .pptx file

"""

Agent 5: Reviewer + Deliverer

python

DELIVERER_PROMPT = """

You are an expert Project Manager & Quality Assurance Lead. Your job is to review all outputs, ensure quality, and deliver everything professionally.

INPUT: Outputs from Agents 1-4

YOUR TASK:

1. Quality Review:

- Check website code for errors
- Verify pitch deck has all 12 slides

- Ensure consistency between brief, website, and deck
- Flag any missing or weak content

2. Generate Pitch Speech Script:

- 5-minute pitch (750-800 words)
- Include timing for each section
- Add natural transitions
- Include Q&A preparation (3 likely questions + answers)

3. Package Everything:

- Website ZIP file
- Pitch deck (.pptx or .pdf)
- Speech script (.txt)
- Final summary document

4. Create User-Friendly Delivery Message:

- Clear instructions on what to do next
- How to deploy website
- How to use pitch deck
- How to practice speech

OUTPUT FORMAT:

```
{
  "quality_score": "8/10 (with improvements made)",
  "improvements_made": ["...", "..."],
  "pitch_speech": "Full 5-minute speech script...",
  "q_and_a": [
    {{question: "...", answer: "..."}}
  ],
  "deliverables": {{
    "website_link": "download link or code",
    "pitch_deck_path": "startup_pitch.pptx",
    "speech_script_path": "pitch_speech.txt",
    "readme_path": "START_HERE.md"
  }},
  "next_steps": [
    "1. Deploy website to Vercel: ...",
    "2. Review pitch deck: ...",
    "3. Practice speech (3 times): ..."
  ]
}
```

3. FULL LANGGRAPH CODE IMPLEMENTATION

Create main.py:

python

from typing **import** TypedDict, Annotated

```

from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
import json
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Initialize LLM (use Groq for speed/cost or OpenAI for quality)
llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.3,
    api_key=os.getenv("OPENAI_API_KEY")
)

# Alternative: Use Groq (faster, cheaper)
# from langchain_groq import ChatGroq
# llm = ChatGroq(model="llama-3.1-70b-versatile", temperature=0.3)

# ===== STATE DEFINITION =====
class AgentState(TypedDict):
    user_idea: str
    polished_brief: str
    research_data: str
    website_code: dict
    pitch_deck_path: str
    speech_script: str
    final_package: dict
    quality_score: str

# ===== AGENT FUNCTIONS =====

def polisher_agent(state: AgentState):
    """Agent 1: Polish the raw idea"""
    print("🪄 Agent 1: Polishing your idea...")

    prompt = ChatPromptTemplate.from_messages([
        ("system", POLISHER_PROMPT),
        ("human", "{user_idea}")
    ])

    chain = prompt | llm
    response = chain.invoke({"user_idea": state["user_idea"]})

    # Parse JSON response

```

```

try:
    polished = json.loads(response.content)
except:
    # Fallback: wrap in JSON
    polished = {"raw_summary": response.content}

return {"polished_brief": json.dumps(polished, indent=2)}

```

```

def researcher_agent(state: AgentState):
    """Agent 2: Deep research"""
    print("🔍 Agent 2: Researching market & competitors...")

    prompt = ChatPromptTemplate.from_messages([
        ("system", RESEARCHER_PROMPT),
        ("human", "Polished brief: {polished_brief}")
    ])

    chain = prompt | llm
    response = chain.invoke({"polished_brief": state["polished_brief"]})

```

```

try:
    research = json.loads(response.content)
except:
    research = {"raw_research": response.content}

return {"research_data": json.dumps(research, indent=2)}

```

```

def website_builder_agent(state: AgentState):
    """Agent 3: Build website"""
    print("💻 Agent 3: Building your website...")

    prompt = ChatPromptTemplate.from_messages([
        ("system", WEBSITE_BUILDER_PROMPT),
        ("human", """
Brief: {polished_brief}
Research: {research_data}
"""))
    ])

    chain = prompt | llm
    response = chain.invoke({
        "polished_brief": state["polished_brief"],
        "research_data": state["research_data"]
    })

```

In real implementation, this would generate actual files

For now, return placeholder structure

```
website_code = {  
    "index.html": "<!-- Generated HTML -->",  
    "styles.css": "/* Generated CSS */",  
    "script.js": "// Generated JavaScript",  
    "README.md": "# Deploy with: vercel deploy --prod"  
}
```

```
return {"website_code": website_code}
```

```
def pitch_deck_agent(state: AgentState):
```

```
    """Agent 4: Generate pitch deck"""
```

```
    print("📄 Agent 4: Creating pitch deck...")
```

```
    prompt = ChatPromptTemplate.from_messages([
```

```
        ("system", PITCH_DECK_PROMPT),
```

```
        ("human", ""
```

```
Brief: {polished_brief}
```

```
Research: {research_data}
```

```
    """)
```

```
])
```

```
    chain = prompt | llm
```

```
    response = chain.invoke({
```

```
        "polished_brief": state["polished_brief"],
```

```
        "research_data": state["research_data"]
```

```
    })
```

In real implementation, use python-pptx to generate actual .pptx

```
pitch_deck_path = "startup_pitch_deck.pptx"
```

```
return {"pitch_deck_path": pitch_deck_path}
```

```
def deliverer_agent(state: AgentState):
```

```
    """Agent 5: Review & deliver everything"""
```

```
    print("📦 Agent 5: Reviewing and packaging delivery...")
```

```
    prompt = ChatPromptTemplate.from_messages([
```

```
        ("system", DELIVERER_PROMPT),
```

```
        ("human", ""
```

```
Polished Brief: {polished_brief}
```

```
Research Data: {research_data}
```

```
Website Code: {website_code}
```

```
Pitch Deck: {pitch_deck_path}
```



```
""")
])
```

```
chain = prompt | llm
response = chain.invoke({
    "polished_brief": state["polished_brief"],
    "research_data": state["research_data"],
    "website_code": str(state["website_code"]),
    "pitch_deck_path": state["pitch_deck_path"]
})
```

```
speech_script = """
5-MINUTE PITCH SCRIPT
=====
```

```
[0:00-0:30] OPENING
Good morning investors. I'm [Name], founder of [Startup].
In 30 seconds: We solve [problem] for [audience] by [solution].
```

```
[0:30-1:30] PROBLEM
[From pitch deck - problem slide]
```

```
[1:30-2:30] SOLUTION
[From pitch deck - solution slide]
```

```
[Continue with all 12 slides...]
```

```
[4:30-5:00] THE ASK
We're raising $X to achieve Y milestones. Join us.
"""
```

```
final_package = {
    "website_files": state["website_code"],
    "pitch_deck": state["pitch_deck_path"],
    "speech_script": speech_script,
    "status": "complete"
}

return {
    "speech_script": speech_script,
    "final_package": final_package,
    "quality_score": "9/10 - Production Ready"
}
```

```
# ===== BUILD WORKFLOW =====
```

```

def build_workflow():
    """Create the LangGraph workflow"""

    workflow = StateGraph(AgentState)

    # Add nodes
    workflow.add_node("polish", polisher_agent)
    workflow.add_node("research", researcher_agent)
    workflow.add_node("build_website", website_builder_agent)
    workflow.add_node("build_pitch", pitch_deck_agent)
    workflow.add_node("deliver", deliverer_agent)

    # Define edges (flow)
    workflow.set_entry_point("polish")
    workflow.add_edge("polish", "research")

    # Research branches to parallel website & pitch
    workflow.add_edge("research", "build_website")
    workflow.add_edge("research", "build_pitch")

    # Both converge to deliver
    workflow.add_edge("build_website", "deliver")
    workflow.add_edge("build_pitch", "deliver")

    workflow.add_edge("deliver", END)

    return workflow.compile()

# ===== RUN THE SYSTEM =====

if __name__ == "__main__":
    # Initialize workflow
    app = build_workflow()

    # User input
    user_idea = input("🚀 Describe your startup idea: ")

    # Run pipeline
    initial_state = {
        "user_idea": user_idea,
        "polished_brief": "",
        "research_data": "",
        "website_code": {},
        "pitch_deck_path": "",
        "speech_script": "",
        "final_package": {},
    }

```

```

    "quality_score": ""
}

print("\n🌀 Starting 5-Agent Pipeline...\n")

result = app.invoke(initial_state)

# Display results
print("\n" + "="*60)
print("✅ YOUR STARTUP BUILDER IS COMPLETE!")
print("="*60)
print(f"\n📊 Quality Score: {result['quality_score']}")
print(f"\n📁 Deliverables:")
print(f"    - Website: {len(result['final_package']['website_files'])} files ready")
print(f"    - Pitch Deck: {result['final_package']['pitch_deck']}")
print(f"    - Speech Script: 5-minute pitch ready")

print(f"\n📝 Polished Brief:")
print(result['polished_brief'][:500] + "...")

print(f"\n🗣️ Pitch Speech Preview:")
print(result['speech_script'][:300] + "...")

print("\n🚀 Next Steps:")
print("1. Save website files and deploy to Vercel")
print("2. Review pitch deck and customize")
print("3. Practice speech 3 times before presenting")

```

4. STREAMLIT UI (Optional but Recommended)

Create app.py:

```

python
import streamlit as st
from main import build_workflow
import json

st.title("🚀 AI Startup Builder - 5-Agent System")
st.write("Enter your startup idea and get: Website + Pitch Deck + Speech")

# Initialize session state
if "result" not in st.session_state:
    st.session_state.result = None

# Input
user_idea = st.text_area("Your Startup Idea (be as messy as you want):",
    height=150,
    placeholder="e.g., An app that helps students find study buddies...")

```

```

if st.button("🚀 Build My Startup", type="primary"):
    if user_idea:
        with st.spinner("👥 Agents are working... This takes 2-3 minutes"):
            app = build_workflow()

            initial_state = {
                "user_idea": user_idea,
                "polished_brief": "",
                "research_data": "",
                "website_code": {},
                "pitch_deck_path": "",
                "speech_script": "",
                "final_package": {},
                "quality_score": ""
            }

            result = app.invoke(initial_state)
            st.session_state.result = result

        st.success("✅ Startup package complete!")

    # Display results
    st.header("📄 Polished Brief")
    st.json(json.loads(result['polished_brief']))

    st.header("🗣️ 5-Minute Pitch Speech")
    st.text_area("Copy this script:", value=result['speech_script'], height=300)

    st.header("📁 Deliverables")
    col1, col2, col3 = st.columns(3)
    with col1:
        st.info("🌐 Website Files")
        st.write(f"{len(result['final_package']['website_files'])} files ready")
    with col2:
        st.info("📊 Pitch Deck")
        st.write(result['final_package']['pitch_deck'])
    with col3:
        st.info("🎯 Quality Score")
        st.write(result['quality_score'])
    else:
        st.warning("⚠️ Please enter your startup idea")

```

Run with: streamlit run app.py

5. QUICK START COMMANDS

bash

```
# 1. Clone or create project
mkdir startup-agent-5
cd startup-agent-5

# 2. Create files
# Copy the code above into:
# - main.py (full workflow)
# - app.py (Streamlit UI - optional)
# - .env (API keys)
# - requirements.txt

# 3. Install
pip install -r requirements.txt

# 4. Add your API key to .env
echo "OPENAI_API_KEY=your_key_here" > .env

# 5. Run (CLI version)
python main.py

# OR (Web UI version)
streamlit run app.py
```

6. WHAT MAKES THIS ROBUST

Feature	Why It's Strong
5 specialized agents	Clear separation of concerns
Parallel execution	Website + Pitch built simultaneously
Quality review step	Agent 5 catches errors before delivery
Polished input	Agent 1 handles messy user input
Research enhancement	Agent 2 adds real market data
Complete delivery	Everything in one package
LangGraph orchestration	Professional state management

NEXT STEPS FOR YOU:

- 1. **Start simple:** Run with mock data first (no API keys)
- 2. **Add one agent at a time:** Test Agent 1 → then add Agent 2, etc.
- 3. **Add real file generation:**

- Use python-pptx for actual .pptx files
 - Write website files to disk
 - Create ZIP for download
4. **Add deployment:** Integrate Vercel API for auto-deploy
 5. **Add RAG:** Use LangChain for research agent to search knowledge base

Would you like me to:

- Generate the python-pptx code for actual pitch deck generation?
- Create the website file generation code ?
- Add RAG implementation for the researcher?
- Create a deployment script?

Just tell me which part you want to build first! 🚀

how rag is even fits here? is it useless or usefull to use rag pipeline here? or any other idea where we can add this for seriously for proper important work?

RAG is NOT useless here — it's actually CRITICAL for making this system production-quality and impressive. Let me explain exactly where and why RAG fits:

🔗 WHERE RAG FITS (And Why It's Essential)

1. RAG is USEFUL for Agent 2 (Researcher)

Without RAG:

- Agent 2 just guesses market sizes, trends, competitors
- Outputs are generic and often wrong
- "The market is \$10B" (made up, no source)

With RAG:

- Agent 2 searches a **knowledge base** of real market research reports
- Pulls actual statistics from trusted sources
- "The TAM is \$12.3B (source: Gartner 2025 Report)"

How it works:

python

Before RAG (weak)

researcher_agent: "I think the market is around \$5-10B"

With RAG (strong)

researcher_agent:

1. Vector search → finds "SaaS Market Report 2025.pdf"
2. RAG retrieval → extracts: "TAM = \$12.3B, CAGR = 18%"
3. LLM synthesizes → "According to Gartner 2025, TAM is \$12.3B"

2. RAG is USEFUL for Agent 1 (Polisher)

Without RAG:

- Just rewrites using LLM's general knowledge
- Doesn't know startup best practices

With RAG:

- Searches knowledge base of **successful startup briefs**
- Imitates proven structures from Y Combinator, Techstars applications
- Uses real templates that investors expect

Example:

```
python
# RAG knowledge base contains:
- 500 YC startup applications (anonymized)
- Best practices for pitch briefs
- Common mistakes to avoid

# Agent 1 uses this to polish:
"user_idea = 'app for students'"
↓ (RAG finds similar successful startups)
"polished_brief = 'StudyBuddy: Peer-to-peer study matching platform for college students (like Tandem for studying)'"
```

3. RAG is USEFUL for Agent 4 (Pitch Deck)

Without RAG:

- Generic 12-slide template
- Doesn't know what investors actually want in 2026

With RAG:

- Searches database of **successful pitch decks** (Sequoia, a16z portfolios)
- Learns what slides worked for funded startups
- Adapts structure based on startup type (SaaS vs e-commerce vs marketplace)

🚫 WHERE RAG IS NOT USEFUL (And What to Use Instead)

Agent	RAG?	Better Alternative
Agent 1 (Polisher)	✅ Useful	RAG on startup briefs
Agent 2 (Researcher)	✅ CRITICAL	RAG + Web Search Tools
Agent 3 (Website Builder)	❌ Not useful	Code generation + Code interpreter
Agent 4 (Pitch Deck)	✅ Useful	RAG on pitch decks
Agent 5 (Deliverer)	❌ Not useful	Quality check rules + LLM judgment

💡 **BETTER IDEA: Where to Add RAG for "Serious Important Work"**

Since you want this to be **impressive for internships/jobs**, here are **3 powerful RAG use cases** that actually matter:

🧠 **IDEA 1: RAG for Personalized Learning (Prefect/Student Context)**

Since you mentioned "student" in your original idea, make this system **actually help students learn**:

```
python
RAG Knowledge Base:
|— Computer Science Fundamentals (CS61B, MIT 6.006 notes)
```

- └─ System Design Interviews (alex Xu books, Grokking)
- └─ AI/ML Papers (Attention Is All You Need, etc.)
- └─ Startup Case Studies (YC library, Sequoia guides)
- └─ Industry Best Practices (Google SRE, AWS docs)

Use Case:

- Student asks: "Build an AI startup for todo lists"
- RAG retrieves:
 - How Notion built their AI features
 - Todo app market saturation data
 - Best tech stack **for** 2026 (**from** real developer surveys)
- Output **is** ****actually useful****, **not** generic

Why this impresses:

- Shows you understand **domain-specific RAG**
- Real educational value for students
- Demonstrates knowledge retrieval + synthesis

🔥 IDEA 2: RAG for Code Quality & Security (Production-Ready)

Add a **6th Agent: Code Reviewer with RAG** that checks security:

python

Agent 6: Security & Quality Reviewer (NEW)

- └─ RAG Knowledge Base:
 - | └─ OWASP Top 10 vulnerabilities
 - | └─ Secure coding patterns (**from** real audits)
 - | └─ Performance best practices (**from** Google/Meta)
 - | └─ Common bugs **in** React/Node.js
- └─ What it does:
 1. Scans generated website code
 2. RAG searches: "XSS vulnerability in React"
 3. Finds secure patterns **from** knowledge base
 4. Fixes vulnerabilities automatically
 5. Outputs: "Fixed 3 security issues, improved performance by 40%"

This **is** **SERIOUS** work because:

- Real startups fail due to security bugs
- Shows you care about production quality
- Demonstrates RAG + Code + Security (rare combo)

🔥 IDEA 3: RAG for Market Validation (Investor-Ready)

This is the **MOST impressive version** — make the system **actually validate if the startup idea is viable**:

python

Enhanced Agent 2 (Researcher) **with** RAG + Web Search:

RAG Knowledge Base:

- └─ 10,000+ failed startup post-mortems (**from** YC, IndieHackers)

- Successful startup **case** studies
- Market research reports (Statista, Gartner, CB Insights)
- Competitor analysis templates
- Customer acquisition cost data by industry

Workflow:

1. User inputs: "AI todo app for students"
2. Agent 2 RAG searches:
 - "todo app startup failures" → finds 47 failed todo apps
 - "student productivity app market" → TAM = \$2.3B
 - "customer acquisition cost SaaS education" → \$45/user
3. Web search **for** latest data (2026 trends)
4. Outputs VALIDATION REPORT:

⚠	WARNING: High Competition	
-	47 similar startups failed	
-	Main reason: No differentiation	
✓	OPPORTUNITY:	
-	Niche: AI-powered study groups	
-	TAM: \$2.3B, growing 22% YoY	
-	CAC: \$45, LTV: \$320 (good ratio)	
💡	RECOMMENDATION:	
	Pivot to: "AI study group matcher"	
	instead of generic todo app	

This **is** SERIOUS because:

- Actually saves students **from** building bad ideas
- Shows business acumen + technical skills
- Real investors use similar analysis

🔗 HOW TO IMPLEMENT RAG (Practical Code)

Here's the **exact RAG setup** for Agent 2 (Researcher):

```
python
# ===== RAG SETUP FOR RESEARCHER =====

from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader, DirectoryLoader

# 1. Create knowledge base (do this once)
def create_knowledge_base():
    """Load market research, startup guides, pitch decks into vector store"""
```

Load documents (PDFs, TXT, MD files)

```
loader = DirectoryLoader("./knowledge_base/", glob="**/*.pdf")
documents = loader.load()
```

Also load web data (scrapped market reports)

```
from langchain_community.document_loaders import WebBaseLoader
web_loader = WebBaseLoader([
    "https://www.ycombinator.com/library/4f-how-to-pitch-to-investors",
    "https://www.sequoiacap.com/article/startup-guide/"
])
documents += web_loader.load()
```

Split into chunks

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(documents)
```

Store in vector database

```
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings()
)
```

return vectorstore

2. Use RAG in Researcher Agent

```
def researcher_agent_with_rag(state: AgentState, vectorstore):
    """Agent 2 with RAG for real market data"""
```

```
polished_brief = json.loads(state["polished_brief"])
startup_type = polished_brief.get("suggested_tech_stack", "SaaS")
```

Query RAG for relevant data

```
retriever = vectorstore.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"k": 5, "score_threshold": 0.7}
)
```

```
relevant_docs = retriever.invoke(
    f"market size {startup_type} competitors 2026 trends"
)
```

Extract context from docs

```
rag_context = "\n\n".join([doc.page_content for doc in relevant_docs])
```

```
# Now use this context in LLM prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", """
    You are a Market Research Analyst. Use the provided context from
    trusted sources to answer accurately. Cite sources when possible.

    CONTEXT FROM KNOWLEDGE BASE:
    {rag_context}

    RESEARCHER_PROMPT: {research_prompt}
    """),
    ("human", "{user_input}")
])

chain = prompt | llm
response = chain.invoke({
    "rag_context": rag_context,
    "research_prompt": RESEARCHER_PROMPT,
    "user_input": state["polished_brief"]
})

return {"research_data": response.content}

# 3. Initialize once at startup
vectorstore = create_knowledge_base()

# Then use in workflow
def build_workflow_with_rag():
    workflow = StateGraph(AgentState)

    workflow.add_node("polish", polisher_agent)
    workflow.add_node("research", lambda state: researcher_agent_with_rag(state, vectorstore))
    workflow.add_node("build_website", website_builder_agent)
    workflow.add_node("build_pitch", pitch_deck_agent)
    workflow.add_node("deliver", deliverer_agent)

    # ... rest of workflow
    return workflow.compile()
```

终极推荐：RAG 的 3 个真正有力的使用场景

场景	难度	impressed 程度	实际价值
Agent 2 市场调研 RAG	☆☆	☆☆☆☆	💰 避免学生做烂项目

场景	难度	impressed 程度	实际价值
Agent 6 代码安全 RAG	☆☆☆	☆☆☆☆☆	🔒 生产级质量
个性化学习 RAG	☆☆	☆☆☆☆	📖 真正帮助学生

 MY FINAL ADVICE

For your project to be **seriously impressive**:

- 1. **Add RAG to Agent 2 (Researcher)** - This is the easiest and most impactful
- 2. **Add a 6th Agent (Code Security Reviewer) with RAG** - This makes it production-grade
- 3. **Build a knowledge base** with:
 - 50+ failed startup post-mortems
 - 20+ successful pitch decks (YC portfolio)
 - Market research reports (PDFs)
 - OWASP security guidelines

This combination shows:

-  Multi-agent orchestration (LangGraph)
-  RAG + retrieval (advanced AI)
-  Code generation (practical)
-  Security awareness (production-ready)
-  Business acumen (market validation)